

FOOD REMEDY API

RE032- Performance Optimization for Mobile Application Deployment

Prepared For: Food Remedy API – Research Team

Author: Sehajpreet Singh

Date: March 2026 (T1 2026)

Table of Contents

Overview	1
Performance benchmarks	1
Key optimisation techniques	2
Common deployed app performance issues.....	3
Platform comparison.....	4
Best practices for Food Remedy app	5
Summary.....	5

Overview

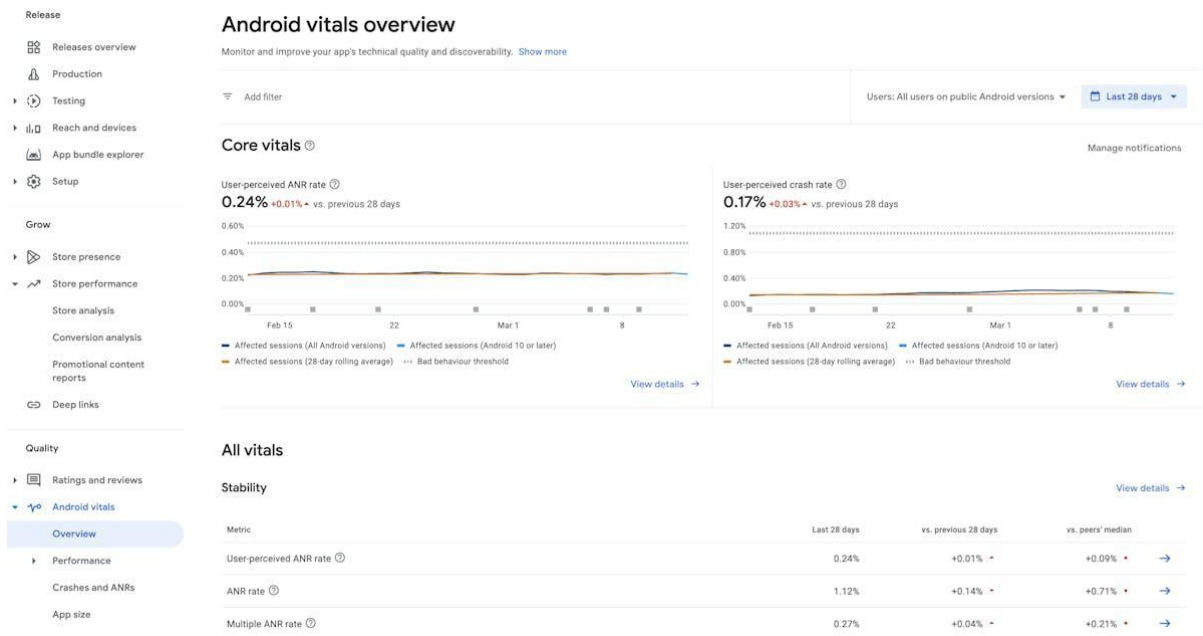
Optimizing performance before deploying a mobile application to production is important because users expect fast loading, smooth navigation and quick responses. And that's especially true for the Food Remedy app, where users might scan products in supermarkets and want instant results for nutrition, allergens and recommendations.

Performance benchmarks

Mobile apps should start quickly, switch screens smoothly, and consume resources sparingly. Android vitals tracks app quality metrics such as startup time, crashes, slow rendering, battery drain and background activity. Google also cites slow rendering and frozen frames as big performance issues.

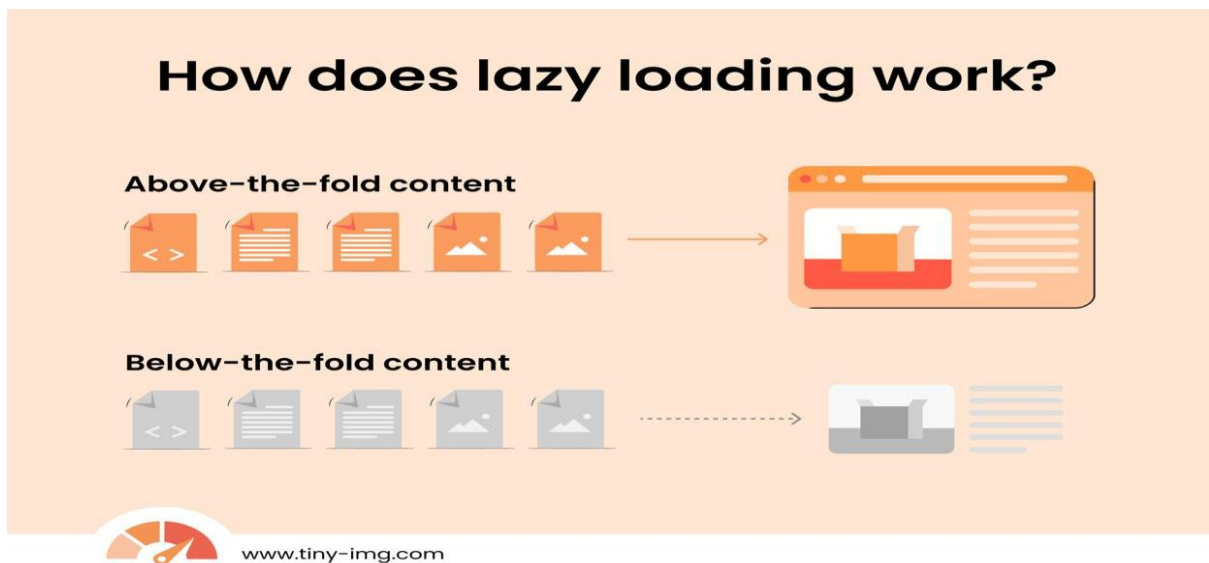
Area	Target standard
App launch time	Ideally under 2 seconds for a smooth first impression
Screen response	User actions should respond immediately or show loading feedback
Frame rendering	Aim for 60 FPS; avoid frames taking longer than 16ms

Frozen frames	Avoid frames taking over 700ms, as they feel stuck
API response	Keep barcode/nutrition requests fast using caching and optimisation
Battery/memory	Avoid unnecessary background work and memory-heavy assets



Key optimisation techniques

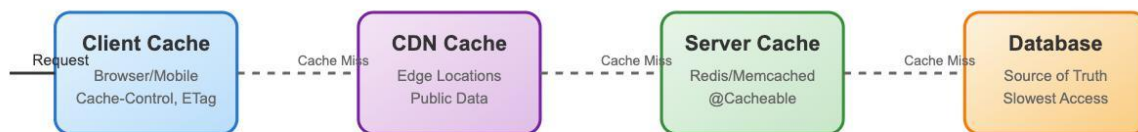
Lazy loading: First load only necessary screens, images and data. For example, the Food Remedy app can load the home screen first, and load detailed nutrition, mood or recommendation modules only when the user opens them. The React Native docs also recommend to delay JavaScript loading until it is needed to reduce startup cost.



Caching: Store frequently accessed data locally, such as recently scanned products, nutrition results, and user preferences. This reduces repeated API calls and improves performance when internet speed is poor.

API optimisation: Remove unnecessary requests, add pagination, compress responses, don't send unused fields, batch related requests where possible. In Food Remedy barcode scanning should first show the most important product information and then later load additional data (like recommendations or alternatives).

API Caching Layers



Optimize images and assets: Compress images, use the right image sizes and don't load large unused files at startup.

Avoid unnecessary re-renders: Rendering the same component multiple times in React Native can block the JavaScript thread. Careful structuring of components with memoization and efficient state management is needed.

Common deployed app performance issues

Issue	Cause	Solution
Slow startup	Too much work during launch	Move non-essential tasks after launch
Laggy UI	Heavy rendering or blocked JS thread	Optimise components and reduce re-renders
Slow API results	Large payloads or repeated requests	Cache, compress, and simplify API responses
High memory use	Large images/assets	Compress and lazy load assets
Battery drain	Background tasks running too often	Limit background sync and wake locks

Mobile Apps Performance



Slow Rendering

Frame >16ms
but <700ms
Jank/Stutter



Frozen Frames

Frame >700ms
Screen
Hung/Frozen



ANR

Main thread
blocked ≥5s
App Not Responding
Dialog

Optimize for Smooth Apps 🚀

Platform comparison

Platform	Optimisation focus
Android	Android vitals, startup time, ANRs, frozen frames, battery usage, background work
iOS	Xcode Instruments, launch time, memory use, responsiveness, avoiding watchdog termination
React Native / Expo	JavaScript bundle size, lazy loading, component re-renders, bridge/native communication, API caching
Flutter	Widget rebuild optimisation, image caching, startup profiling, reducing build/layout cost

Apple also recommends that you reduce app launch time and memory usage, because this improves the user experience and reduces the risk of termination by the system.



Best practices for Food Remedy app

Below are the key performance improvements needed for the Food Remedy app:

- lazy loading on product details pages, recommendation screens and large research-based content.
- Cache now stores recent barcode scans so scanning the same barcode again is instant.
- Only return the product and nutrition fields that are necessary to improve Firebase/API calls.
- Compress images and don't load unnecessary assets at start up.
- Show loading indicators or skeleton screens while fetching product data.
- Test on real Android/iOS devices, not just emulators.
- App store launch time, crashes, slow screens, memory usage, etc. Watch for
- Limit non-essential background processes to conserve battery.
- Keep recent scans and user preferences stored offline.

Summary

Performance optimisation makes the mobile app production ready in terms of speed, responsiveness, stability and resource efficiency. Optimisation is especially important for Food Remedy because when users are shopping they need to be able to scan barcodes quickly, get nutrition results quickly and have a smooth experience with personalised recommendations. The best approach is to do lazy loading, caching, API optimisation, asset compression and real-device testing before deployment.